Testing SpringBoot + PostgreSQL + Docker

1. Start the SpringBoot Application

- Run the Spring Boot application from IntelliJ.

```
Spring Boot Application
        ↓
JPA / Hibernate
        ↓
PostgreSQL
        ↓
Docker Container
```

2. Check the Application Startup

- Look at the SpringBoot console/logs
- If the application starts successfully with JPA/Hibernate, it indicates that the application was able to communicate with the database
- If there is a database authentication/connection error during the startup, check the PostgreSQL connection details

3. Common Windows Issue

You may have:

```
PostgreSQL installed directly on Windows
              +
PostgreSQL running inside Docker
```

Both can potentially use the same PostgreSQL port:

```
5432
```

This can cause connection/authentication problems.

4. Check Windows Services

1. Open the Windows **Start Menu**
2. Search for **services**
3. Open the Services
4. If a locally installed **PostgreSQL service** is still running
    1. Right-click
    2. Stop it

5. Verify PGADMIN

Open:

```
http://localhost:5050
```

Then navigate to:

```
Servers
   ↓
Postgres
   ↓
Databases
   ↓
ecom_db
   ↓
Schemas
   ↓
public
   ↓
Tables
```

6. Check the Tables

After the Spring Boot application starts:

- Hibernate/JPA creates the tables corresponding to your application entities.
- Under:

```
ecom_db → Schemas → public → Tables
```

you should see your application's tables.

For example:

```
users
products
cart
orders
...
```

The exact tables depend on the entities in your application.

7. Initially, There May Be No Data

If you open the `users` table and view its data:

- The table exists.
- But it may contain **no records yet**.

That's expected because we haven't created any users through the application.

```
Table exists ✅
Data exists ❌
```

8. Create Data using Postman

Now use **Postman** to call your Spring Boot REST APIs.

For example:

```
Postman
   ↓
Create User API
   ↓
Spring Boot
   ↓
JPA / Hibernate
   ↓
PostgreSQL
```

- Send the **Create User** request.
- The application should return a successful response.

9. Verify the user

Go back to pgAdmin.

Open the `users` table and execute/refresh the data query.

You should now see the newly created user.

```
Postman
   ↓
POST /users
   ↓
Spring Boot
   ↓
PostgreSQL
   ↓
users table
   ↓
New user ✅
```

10. Test Products

You can perform the same test for products.

Flow

```
Postman
   ↓
Create Product API
   ↓
Spring Boot
   ↓
PostgreSQL
   ↓
products table
```

- Create a product using Postman.
- Go to pgAdmin.
- Open the `products` table.
- Execute/refresh the query.
- Verify that the product has been inserted.

11. Check SQL Queries

If you configured:

YAML

```yaml
spring:
  jpa:
    show-sql: true
```

Hibernate will print generated SQL statements in the Spring Boot console.

For example:

SQL

```sql
insert into products ...
```

So you can observe:

```
Postman Request
      ↓
Spring Boot API
      ↓
JPA / Hibernate
      ↓
Generated SQL
      ↓
PostgreSQL
```

12. Final Architecture



🧠 Most important takeaway

> If your Spring Boot application starts successfully and JPA/Hibernate can initialize against PostgreSQL, your application has successfully connected to the PostgreSQL database running in Docker. You can then verify the connection by creating data through Postman and checking that the corresponding records appear in pgAdmin.